L2 · PCB DESIGN

- **75-90% of PCB errors are mechanical** (hole size/alignment, footprints, clearances, connectors). Fix: latest data sheets, measure real parts, check pinouts, **multiple independent design reviews**.
- Labels: **R** resistor · **C** capacitor · **Q** transistor · **U** IC · **J** connector.
- Flow: understand circuit → schematic → label → **place parts first** (don't wire yet) → route → review.
- Design for test: sockets, test points (esp. ground), clock access, probe room, wide traces/pads.
- Use **4+ layers**: signals top/bottom, power + ground inner planes. Planes = low-impedance supply/return, less over/undershoot, add C, lower L.
- **Bypass capacitors:** a fast logic edge needs instant current to charge its load; the cap stores charge next to the IC and supplies it. About **1 per 1-2 ICs** (0.1 µF) + a bulk cap (10 µF).
- **Split ground planes:** digital is noisy (switching spikes), analog needs a quiet ground, so separate planes joined at **ONE point** (more joins = ground loops, hum, oscillation).
- **Clock skew:** same clock edge reaches parts at different times (path length, propagation delay, buffers) → eats timing margin.

Trace resistance

$R = \rho L / A$ A = width × thickness $V = IR$ $P = I^2R$

Convert to one unit first: 1 mil = 0.001 in = 0.00254 cm; 1 in = 2.54 cm.

> **EXAMPLE** 1 mil thick, 125 mil wide, 3 in long, 10 A, ρ = 1.68e-6 Ω·cm: A = 125e-6 in² = 8.06e-4 cm²; L = 7.62 cm; R = 15.9 mΩ; V = IR = 0.159 V; P = 1.59 W.

Clocks & pulses

- **f = 1/T.** 1 ms = 1 kHz · 1 µs = 1 MHz · 1 ns = 1 GHz · 2 ns = 500 MHz.
- Prefixes: T 10^{12} · G 10^{9} · M 10^{6} · k 10^{3} · m 10^{-3} · µ 10^{-6} · n 10^{-9} · p 10^{-12} · f 10^{-15}
- **Duty cycle** = $t_{high} / (t_{high} + t_{low}) \times 100\%$. 50% means high time = low time.
- Pulse width: measured at **50%** of max. Rise time: **10→90%**. Fall: 90→10%.
- **Overshoot** above the rail, **undershoot** below ground, **ringing** = damped oscillation. Causes: poor grounding/bypassing, impedance mismatch, inductive loads, fast edges.
- **Propagation delay:** gates need time to switch (~2 ns per foot of PCB), so everything must fit in one clock period — why we can't just raise the clock.

L3 · MICROPROCESSORS

- **µP** = CPU only; needs external memory + I/O (68000, Z80, ARM, RISC-V). **µC** = µP + on-chip peripherals (ports, A/D, D/A, timers, watchdog, RTC): Arduino, Pico, TIVA.
- **Address bus:** one-way (CPU → memory); n lines address **2^n bytes**. **Data bus:** two-way. **Control/status bus:** mixed (RESET, INTERRUPT, READ, WRITE, ADDRESS VALID); each may be in, out or both.

CISC (68000)	RISC (ARM)
many instructions & modes	few instructions
variable length (2-10 bytes)	fixed length, 1 cycle each
memory-to-memory ops	many general registers
compact code, many cycles	register-only math; LOAD / STORE
von Neumann (one memory)	Harvard OK (separate code/data)

Blocks inside a processor

- **State machine** (microcode): decodes instructions, sequences reads/writes, why µPs are **synchronous** (clocked).
- **ALU:** math/logic, results set the flags. **Registers:** fast local storage.
- **Status register** (flags) · **PC** (next instruction address) · **SP** (top of stack, LIFO) · **Instruction register** (holds fetched op-code).
- **Interrupt controller** · **Bus interface unit** (memory/IO timing) · **Debug core** · **Cache** (I and D) · **Glue logic** (NAND/NOT between blocks).

	68000	Z80
size	8-16-32 bit CISC, 1979	8-bit, 1976 (8080 code-compat.)
address	24 lines = 16 MB	16 lines = 64 KB
operand order	**source, dest**	**dest, source**
hex	$FF %bin @oct	0FFH (leading 0 if A-F)
flags	X N Z V C	S Z H P/V N C
multi-byte	big-endian	16-bit little-endian

Registers

- **68000:** D0-D7 data (byte/word/long); A0-A6 address + **A7 = SP** (user + supervisor); PC; SR (CCR flags). Address regs hold only word/long. 32-bit internal, 16-bit external bus.
- **Z80:** A accumulator (all arithmetic) + F; B C D E H L (8-bit; pairs BC DE HL = 16-bit); alternates A' F' B'...L' (EXX); IX IY index; SP; PC; I R.
- **Flags:** C carry · V overflow · Z zero · N negative · X extend (68000 only) · H half-carry, N=subtract (Z80 only).

Two's complement

- Invert all bits, add 1: $5E = 0101 1110 → 1010 0001 + 1 = **$A2**. 8-bit: −128 to +127; 16-bit: −32768 to +32767. Subtraction = add the negative. Overflow only when operands share a sign and the sum won't fit.

L4 · ASSEMBLY LANGUAGE

- The **assembler** turns mnemonics into machine code. Instruction = **operation** (verb) + **operands** (nouns). 68000: at least 16 bits (op-code word) + extension words (immediate/address, high word first). Z80: op-code byte(s) + operand bytes.
- Line format: `LABEL OPCODE OPERAND1,OPERAND2 * comment` (label optional, column 1; comment starts with * or ;).
- **Machine code:** NOP $4E71 · MOVE.B D0,D3 $1600 · ADD.L D4,D7 $DE84 · ADDI.W #$4000,D5 $0645 $4000 · Z80 ADD A,22H = C6 22.

Directive	68000	Z80
origin	ORG	ORG
equate	EQU	EQU
end of source	END	END
define byte / word	DC.B / DC.W	DEFB / DEFW
reserve space	DS.B nn / DS.W	DEFS nn / N/A
ASCII string	DC.B 'string'	DEFM 's'

Also (68000): SET = redefinable EQU · DCB = block of one value · REG = MOVEM list · MACRO · OPT. **EQU** = like #define.

- **MOVE.B D0,D1** (68000) copies D0 → D1, source unchanged; **LD B,A** (Z80) copies A → B. Size suffix **.B .W .L** = 8/16/32 bits. Z80: LD A,C · LD A,(1234H) · LD (HL),A.

> **ZERO A BLOCK** (68000) `LEA start,A0 · LEA end,A1 · loop: CMPA.L A1,A0 · BEQ done · CLR.L (A0)+ · BRA loop · done: SIMHALT`. CLR.L (A0)+ clears 4 bytes and advances A0 by 4. SIMHALT is a simulator command, not a 68000 instruction.

> **LARGEST OF FOUR BYTES** `MOVE.B (A0)+,D1 · CMP.B D1,D0 · BGE keep · MOVE.B D1,D0 · keep:` (D0 = max so far; BGE = signed). Z80: CP [HL] + JR NC (unsigned carry test), DJNZ counts.

Flow chart: rectangle = operation, diamond = decision, arrow = flow; draw before coding.

L5 · 68000 ADDRESSING, FLOW, STACK

The 6-bit effective address (EA) = **3-bit mode + 3-bit register** (8 modes × 8 regs). Each instruction allows only some modes (MOVE.W D0,A1 is illegal; use MOVEA, which changes no flags).

mode	syntax	effect
0	Dn	data register direct
1	An	address register direct (A7 = SP)
2	(An)	An holds the **pointer**; operand is memory there
3	(An)+	use address, **then** An += size (1/2/4)
4	-(An)	An -= size, **then** use address
5	d16(An)	An + sign-extended 16-bit displacement
6	d8(An,Xn)	An + Xn + sign-extended 8-bit disp.
7	abs.W / abs.L	16-bit (sign-ext.: $0-$7FFF, $FF8000-$FFFFFF) / full 32-bit address
7	#imm	number in the instruction (source only)
7	PC-rel	(PC)+d16 / (PC)+Xn+d8; relocatable

- **Stack grows high → low.** PUSH = `MOVE x,-(SP)` · POP = `MOVE (SP)+,x`. Set SP to top of RAM first. Z80 stack also grows down; PUSH/POP use 16-bit pairs.
- Subroutines: **JSR** pushes the 32-bit return address (SP – 4) and jumps; **RTS** pops it; **BSR** = relative call. Save registers: `MOVEM.L list,-(SP)` … `MOVEM.L (SP)+,list`. Z80: CALL / RET (16-bit PC).
- **Jumps:** JMP absolute; BRA relative (±32K); Z80 JR ±128, JP cc, CALL cc, RET cc (68000 has none of those *cc* forms), RTE / RETI return from interrupt.

2-D ARRAY 50 × 50 longs at $4000, element (row 23, col 11): D0 = 23×50×4 = 4600 = $11F8; offset = 11×4 = 44 = $2C; `MOVE.L $2C(A0,D0),D3` with A0 = $4000.

Compare & branch

`CMP src,dst` computes **dst – src**, sets flags, stores nothing. Branch right after. **Z**=1 equal; **N**=1 result negative; **C**=carry/borrow; **V**=signed overflow; **X**=extend.

after CMP D0,D1	signed	unsigned
D1 < D0	BLT	BCS
D1 ≤ D0	BLE	BLS
D1 = D0	BEQ	BEQ
D1 ≠ D0	BNE	BNE
D1 > D0	BGT	BHI
D1 ≥ D0	BGE	BCC / BHS

- Addresses are **unsigned** → use BHS/BCC. BMI: N=1. BPL: N=0. Test bit n: AND with a mask, check Z.
- **FOR/WHILE:** test at top, BRA back at bottom. **DO-WHILE:** body first, test + branch back at bottom (runs ≥1 time). Z80: `CP r`, `DJNZ` (B–1, jump if ≠0), `BIT n,r` (Z = ~bit).

L6 · BUSES, MEMORY CELLS, PAGING

- Plain outputs can't share a bus (contention = short circuit). **Open collector (OC):** output only pulls low; a pull-up resistor makes high; fast falling edge, slow rising edge (RC): t_{rise} (10-90%) = **$2.2 \cdot R \cdot C_{total}$**.
- **Tri-state:** each output has an enable switch; one device drives (enabled), the others are **Hi-Z**.
- **~CE / ~CS** = master control: if high, the whole chip is off (outputs Hi-Z, clocked inputs ignored). **~OE** only puts outputs in/out of Hi-Z. Chip drives the bus only when **~CE = 0 and ~OE = 0**; ~WE low = write.
- **Active low:** ~NAME is true when low. Negative logic swaps gates (De Morgan): invert all inputs/outputs, **OR ↔ AND**, NOR ↔ NAND. NOR is not the negative-logic OR.
- **SRAM cell (6T):** 2 cross-coupled inverters (4 T) hold the bit by feedback + 2 access transistors to BIT/~BIT. Flip-flop cell = D flip-flop + tri-state output.
- **Address space** = 2^n bytes for n lines (capability, not installed memory). Memory is addressed in bytes. Word (16 b) on even addresses; long (32 b) on multiples of 4; double on multiples of 8. Data path width (8/16/32/64) sets performance.
- **Endian:** big = most-significant byte at the lowest address (68000). Little = least-significant byte first (Z80 16-bit values).
- **Byte strobes:** chip lane per byte; WR0..WR3 pick lanes: byte = 1 strobe, word = 2, long = all 4. A0 (and A1) are implied by the strobes.

L7 · MEMORY DECODING & TIMING

offset bits = $\log_2$(chip depth) page bits = address lines – offset pages = $2^{\text{page bits}}$
chips/page = bus width ÷ chip width

- **Offset** (low) bits go straight to every chip's address pins. **Page** (high) bits + address-valid go to the **decoder**, whose outputs are the active-low ~CE of each page.
- **Page range:** start = n × page size; end = start + size – 1. Easy way: write the page number, then offset-bit-count zeros, regroup in hex.
- **Chip select as a NAND:** output low only if every input is 1; invert each page bit that must be 0; include address-valid. Page $1A4 (A23-A15 = 1 1010 0100): inverters on A21, A19, A18, A16, A15.
- **Design steps:** (1) chips/page & page size, (2) offset vs page bits, (3) which pages exist and where, (4) decode each page uniquely.
- **74x138** (3-to-8, outputs active low): on when EN1 = 1, ~EN2 = ~EN3 = 0. ABC pick Y0-Y7. A23-A21 with ~AS on ~EN2 = 8 regions of 2 MB (16 MB map). Second 138 + one more bit = 16.
- **Memory map (68000):** vectors at $0, then ROM, peripherals, empty space, RAM at top. Paging extends a small address space: a register picks which 64 KB page is swapped in.

given	offset	page	pages	chips
20-bit, 8-b bus, 256K×4	A0-17	A18-19	4	8
24-bit, 8-b bus, 512K×8	A0-18	A19-23	32	32
24-bit, 8-b bus, 256K×4	A0-17	A18-23	64	128
68000, 32K pages	A0-14	A15-23	512	—

Timing & DRAM

- Data valid = **decoder delay + access time** (from ~CE low). Must fit the CPU's read window, else add **wait states** (+1 clock each) or a faster part. Z80 timing is synchronous; ~WAIT stretches it.
- **Max clock, zero waits:** $N \cdot T \geq (\text{gates} \times t_{gate}) + t_{access}$. HW4: 2 gates × 2 ns + 15 ns = 19 ns over 3 clocks → T = 6.33 ns = **158 MHz**.
- **SRAM:** 6T, no refresh, fast (cache). **DRAM:** 1T + C, denser, must refresh (~10-16 ms), multiplexed row/column address: 64 Mbit = 8192^2 = 13 + 13 bits. Read: row, ~RAS↓, column, ~CAS↓, data. Write: ~WE low, data before ~CAS↓. Refresh = RAS cycle per row (~2 µs each).

HOMEWORK 2-4 & PRACTICE ANSWERS

- **HW2 trace:** $4F, $17 stored by (A0)+ → word $4F17; AND.W with $5555 = **$4515** ($45 at $4000, $15 at $4001).
- **HW3:** add longs `MOVE.L a,D0` / `ADD.L b,D0` / `MOVE.L D0,r`. Search $AA55: `CMP.W (A0)+,D0` / `BNE loop` / `LEA -2(A0),A2`, BHS ends. Smallest byte: signed start $7F + BGT; unsigned $FF + BHI.
- **HW4:** ROM 128K: A19-17 = 000 / 001. RAM 64K: A19-16 = 1000 / 1001. 32-bit CPU A2-A31, 2M×8: 4 GB, **512 pages, 2048 chips**, page bits A23-31. 16 pages with two 138s: A20-22 on ABC of both; A23 → ~EN3 (first), EN1 (second).

P1	ρ(77 K) = 5.67e-9·77 + 1.58e-8 = 4.524e-7 Ω·cm; R = 0.1/8 = 0.0125 Ω; A = ρL/R = 3.677e-4 cm²; W = A/t = 0.0724 cm = **28.5 mil**
P2	a F · b F (500 MHz) · c ~T (CLR.B sets Z; LD C,0 sets no flags) · d F (66.7%) · e F (OR→AND) · f T · g T · h T
P3	**2** chips/page, **128** chips, offset A0-17, page A18-23, **64** pages; $000000-03FFFF, $040000-07FFFF, $080000-0BFFFF, $0C0000-0FFFFF
P5	BEQ↔JR Z · CMP.B D0,D1↔CP D · MOVE.B #$55,D7↔LD C,0A4H · MOVE.B D1,$FF35↔LD [0A3E7H],A · MOVE.B D6,D5↔LD B,A
P6	36FC 3400 · DD39 A457 8A04 · B786 · 4846 · 4279 AD45 9BC0 (abs.long = mode 7 reg 1; SWAP = $4840 + n)

P7 COUNT 1 BITS `MOVE.L $2000,D0 · CLR.L D1 · MOVE.W #32,D2 · loop: LSL.L #1,D0 · BCC skip · ADDQ.B #1,D1 · skip: SUBQ.W #1,D2 · BNE loop` (top bit → C each shift).

COMMON TRAPS

- .W on longword data adds only the low word — match the size to the data.
- `CMP src,dst` = dst – src; signed (BGT/BLT) vs unsigned (BHI/BCS).
- (An)+ moves by operand size (1/2/4), not always 1.
- Simulator shows bytes: word $4515 = $45 then $15.
- Pages use chip **depth**; chips per page use chip **width**.
- Wait-state math: use the stated clocks per access (3), not 1.